# NAND TO TETRIS

NAME: - Shresth Priyadarshi

ROLL NO.: - 2501410026

Project 2

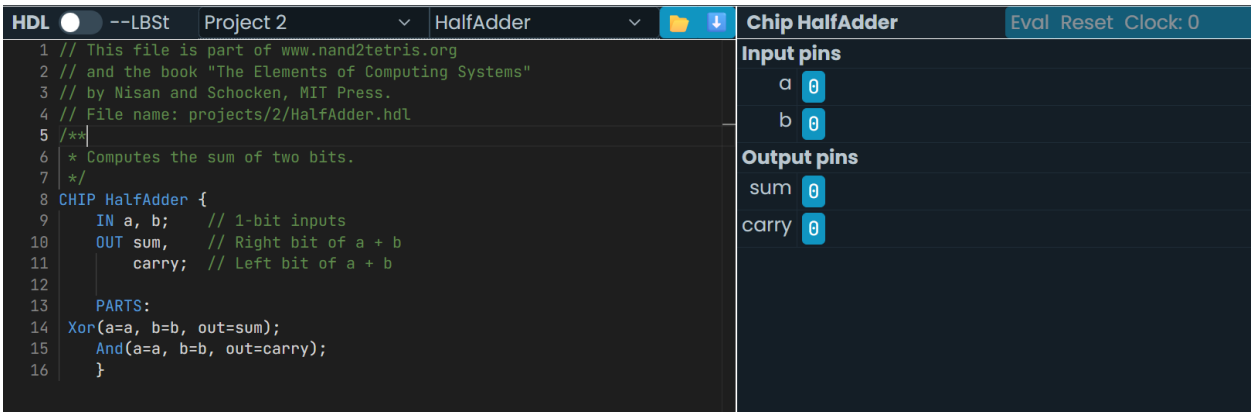
## Half Adder

Code

```
CHIP HalfAdder {
    IN a, b;    // 1-bit inputs
    OUT sum,    // Right bit of a + b
        carry;  // Left bit of a + b

    PARTS:
Xor(a=a, b=b, out=sum);
    And(a=a, b=b, out=carry);
    }
```

Screenshot



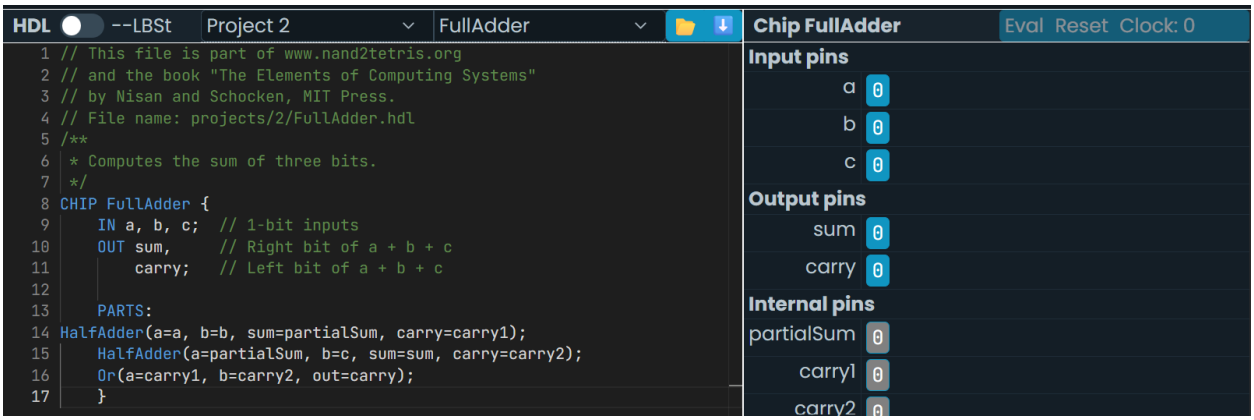

# Full Adder

## Code

```
CHIP FullAdder {
    IN a, b, c;  // 1-bit inputs
    OUT sum,     // Right bit of a + b + c
        carry;   // Left bit of a + b + c

    PARTS:
HalfAdder(a=a, b=b, sum=partialSum, carry=carry1);
    HalfAdder(a=partialSum, b=c, sum=sum, carry=carry2);
    Or(a=carry1, b=carry2, out=carry);
    }
```

## Screenshot




# Add16

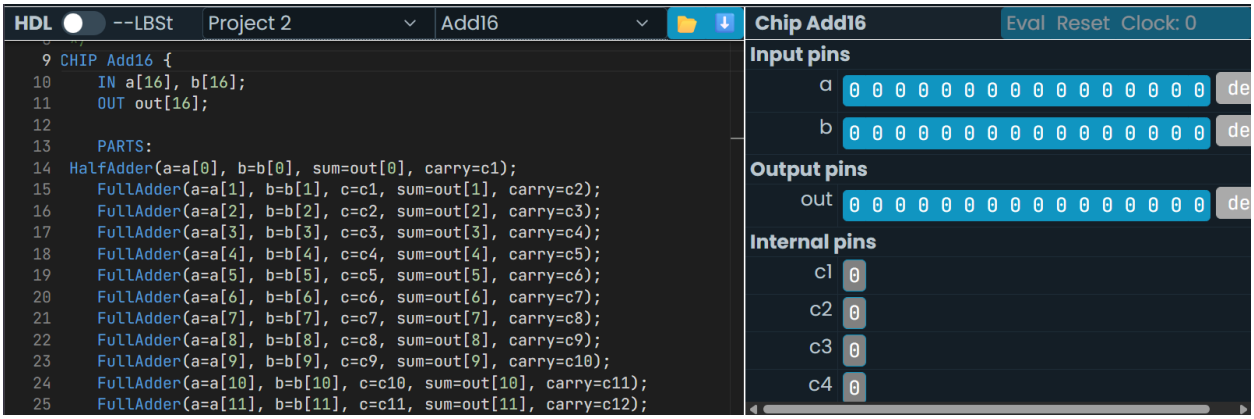
## Code

```
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
 HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c1);
    FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
    FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
    FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
    FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
    FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
    FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
    FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c8);
    FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c9);
    FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c10);
    FullAdder(a=a[10], b=b[10], c=c10, sum=out[10], carry=c11);
```

```
    FullAdder(a=a[11], b=b[11], c=c11, sum=out[11], carry=c12);
    FullAdder(a=a[12], b=b[12], c=c12, sum=out[12], carry=c13);
    FullAdder(a=a[13], b=b[13], c=c13, sum=out[13], carry=c14);
    FullAdder(a=a[14], b=b[14], c=c14, sum=out[14], carry=c15);
    FullAdder(a=a[15], b=b[15], c=c15, sum=out[15], carry=overflow);
    }
```

## Screenshot




# ALU

## Code

```
CHIP ALU {
    IN
        x[16], y[16],  // 16-bit inputs
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f,  // compute (out = x + y) or (out = x & y)?
        no; // negate the out output?
    OUT
        out[16], // 16-bit output
        zr,      // if (out == 0) equals 1, else 0
        ng;      // if (out < 0)  equals 1, else 0

    PARTS:

     Mux16(a=x, b=false, sel=zx, out=x1);
    Not16(in=x1, out=nx1);
    Mux16(a=x1, b=nx1, sel=nx, out=x2);

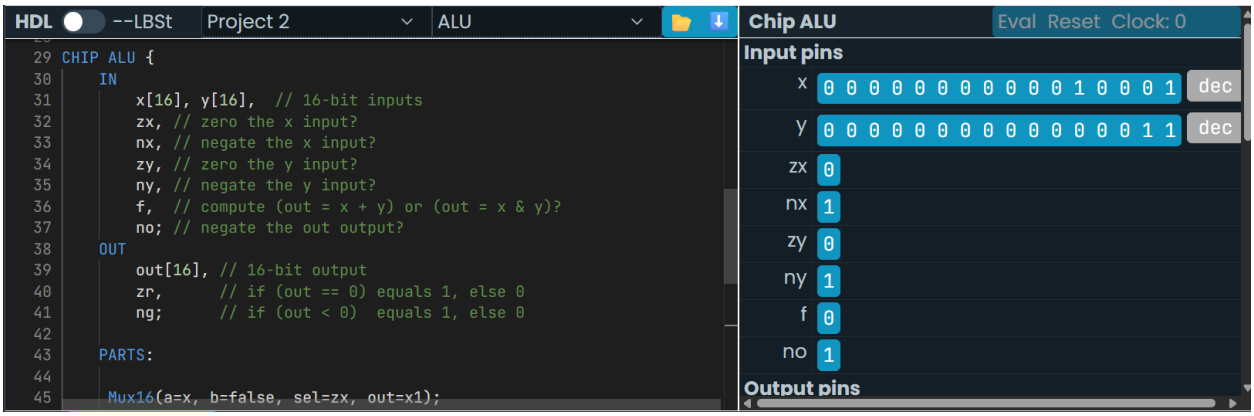
    Mux16(a=y, b=false, sel=zy, out=y1);
    Not16(in=y1, out=ny1);
    Mux16(a=y1, b=ny1, sel=ny, out=y2);

    And16(a=x2, b=y2, out=fAnd);
    Add16(a=x2, b=y2, out=fAdd);
    Mux16(a=fAnd, b=fAdd, sel=f, out=fOut);

    Not16(in=fOut, out=nfOut);
    Mux16(a=fOut, b=nfOut, sel=no,
          out=out,
          out[0..7]=low8,
          out[8..15]=high8,
          out[15]=ng);

    Or8Way(in=low8, out=orLow);
    Or8Way(in=high8, out=orHigh);
    Or(a=orLow, b=orHigh, out=orAny);
    Not(in=orAny, out=zr);
}
```

Screenshot


HDL
--LBSt
Project 2
ALU
CHIP ALU {
IN
x[16], y[16], // 16-bit inputs
zx, // zero the x input?
nx, // negate the x input?
zy, // zero the y input?
ny, // negate the y input?
f, // compute (out = x + y) or (out = x & y)?
no; // negate the out output?
OUT
out[16], // 16-bit output
zr, // if (out == 0) equals 1, else 0
ng; // if (out < 0) equals 1, else 0
PARTS:
Mux16(a=x, b=false, sel=zx, out=x1);
Chip ALU
Eval Reset Clock: 0
Input pins
x 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1
dec
y 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1
dec
zx 0
nx 1
zy 0
ny 1
f 0
no 1
Output pins